

Documentation

Integrations & API Testing

Using Postman

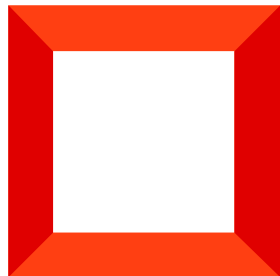
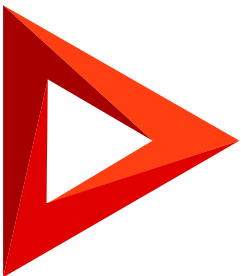


Table of Contents

Using Postman.....	1
1. Overview	5
1.1 General Information	5
1.2 Authentication	5
1.3 Response Format.....	5
1.4 HTTP Status Codes	6
2. Health Check	6
3. Authentication	7
3.1 Login	7
Request Body	7
Request Example (SOC Manager)	8
3.2 Verify Token (Get Current User)	8
4. Alerts	9
4.1 Create Alert	9
Request Example	9
4.2 Get Alert Statistics	9
4.3 Create Alert	10
Request Body	10
4.4 Acknowledge Alert	11
Path Parameters	11
5. Incidents.....	12
5.1 Get All Incidents	12
5.2 Create Incident.....	12
Request Body	12
5.3 Update Incident	13
Path Parameters	13
Request Body	13
Request Example	14
6. Threat Intelligence.....	14
6.1 Scan IOC	14
Request Body	14
Request Example	15
6.2 Get Recent Threat Intelligence.....	15
Request Example	15
6.3 Block IP / Entity	16

Request Body	16
Request Example	16
7. Geo-Location Lookup	16
7.1 Single IP Lookup	17
Path Parameters	17
Request Example	17
7.2 Bulk IP Lookup	17
Request Body	17
Request Example	18
8. Reports	18
8.1 Report Summary (JSON)	18
Request Example	18
8.2 Alert Report — PDF	19
Response Headers	19
Request Example	19
8.3 Alert Report — CSV	19
Query Parameters	20
CSV Report Structure	20
Request Example	20
9. Dashboard	21
9.1 Dashboard Summary	21
9.2 Recent Alerts	21
10. User Management	22
10.1 Get All Users	22
Access Control	22
10.2 Create User	23
Request Body	23
10.3 Register Viewer	24
10.4 Login Viewer	25
10.5 Incidents Access by Viewer	25
10.6 Get Users by Viewer	26
10.7 Delete Incidents by Viewer	26
11. Viewer Role — Access Restrictions	27
12. Analyst Role Login	28
12.1 Analyst Get Alerts	28
12.2 Analyst Get Incidents	29

12.3 Analyst Add Incident.....	30
12.4 Analyst Update Incident.....	30
12.5 Analyst Get Summary	31
12.6 Analyst Delete Incident.....	32
12.7 Analyst Get Users	32
12.8 Analyst Block IP	33
13 Error Handling	34
13.1 No Token Provided.....	34
13.2 Invalid Id	35
13.3 Login with Invalid Credentials	35
13.4 Wrong Password Attempts.....	36
13.5 SQL Injection.....	36
13.6 XSS Attack.....	37
14 Time Based Blind SQL.....	37
Quick Detection:.....	38
15. Roles & Permissions Summary.....	38
16. Quick Reference — All Endpoints	39

REST API Documentation

Using Postman

Postman is a toolset for API testing. It is a development environment that lets you create, test, manage, and publish API documentation.

Base URL: `http://localhost:5000`

1. Overview

LogEase SIEM is a Security Information and Event Management platform. This document describes all REST API endpoints, including Authentication, Alerts, Incidents, Threat Intelligence, Geo-lookup, and Reports.

1.1 General Information

Base URL	http://localhost:5000
Content-Type	application/json
Authentication	Bearer Token (JWT)
API Prefix	/api

1.2 Authentication

Most endpoints require a Bearer Token in the Authorization header:

Authorization: Bearer <jwt token>

1.3 Response Format

All responses follow this standard JSON envelope:

```
{  
  "success": true | false,  
  "message": "Human-readable message",  
  "data": {...}  
}
```

1.4 HTTP Status Codes

Status Code	Meaning
200 OK	Request succeeded
201 Created	Resource created successfully
400 Bad Request	Invalid request body or parameters
401 Unauthorized	Missing or invalid JWT token
403 Forbidden	Insufficient permissions
404 Not Found	Resource does not exist
500 Internal Server Error	Server-side error

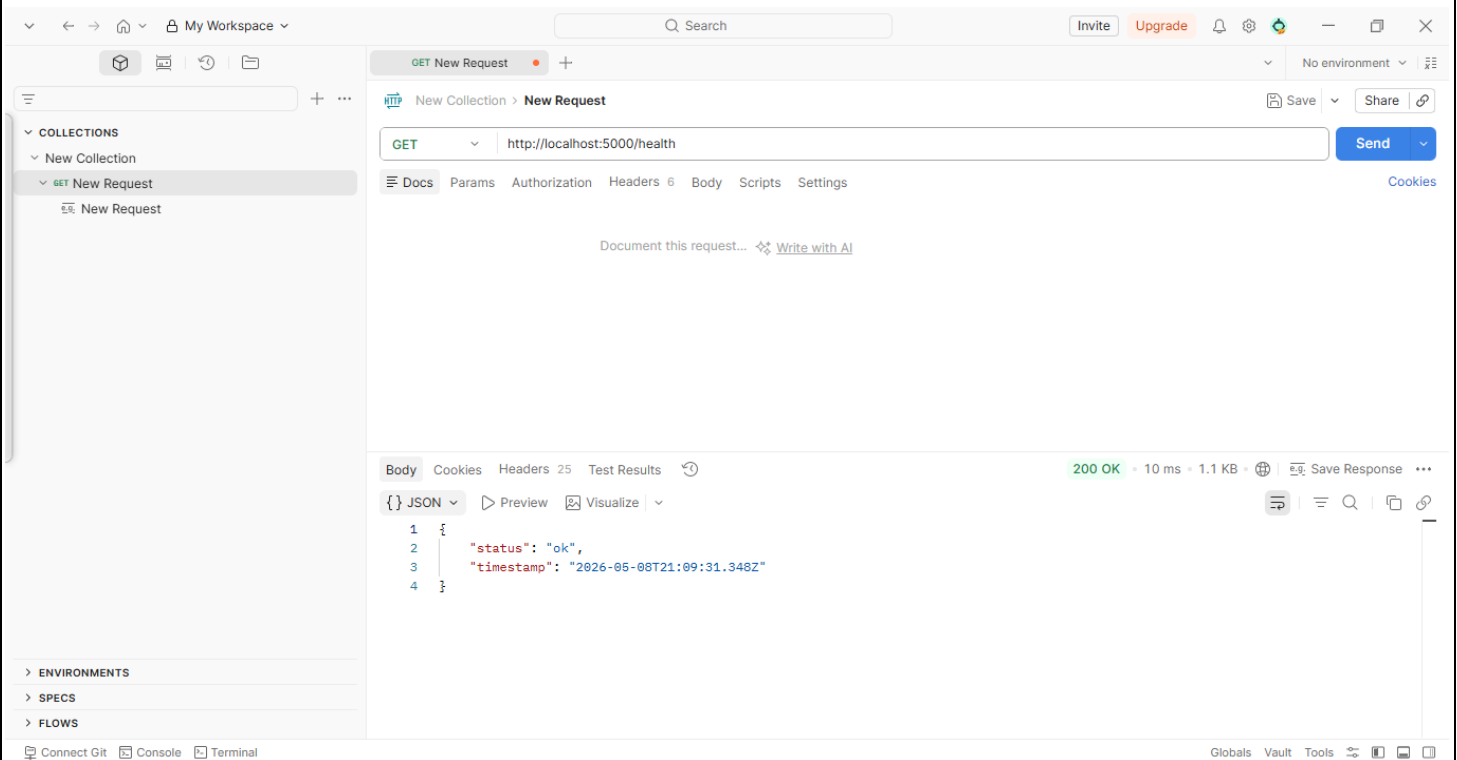
2. Health Check

Verify that the backend server is running.

GET	/health	Server health status
-----	---------	----------------------

No authentication required.

Response Example



3. Authentication

Endpoints for user registration, login, and token verification.

3.1 Login

POST

/api/auth/login

Authenticate user and obtain JWT

Request Body

Field	Type	Required	Description
email	string	Required	User's email address
password	string	Required	User's password

Request Example (SOC Manager)

Postman interface showing a POST request to `http://localhost:5000/api/auth/login`. The request body is a JSON object:

```
1 {
2   "email": "manager@soc.com",
3   "password": "Manager123!"
4 }
```

The response is a 200 OK status with a JSON body:

```
2 {
3   "success": true,
4   "message": "Login successful",
5   "data": {
6     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJ1c2VybmFtZSI6InV19tYW5hZ2VYIiwiaWwiOiJhbmwiOiJtYW5hZ2VYQHNVy5jb28iLCJyb2xlIjoic29jX21hbmFnZXIiLCJpYXQiOiJlbnRzgyNzUyNjAsImV4cCI6MTc3ODMwNDA2MH0.1iFraw5L358IQuFhc1Y3H1_3MYaaltxAjj08mhUpqes",
7     "user": {
8       "user_id": 2,
9       "username": "soc_manager",
10      "email": "manager@soc.com",
11      "role": "soc_manager"
12    }
13  }
14 }
```

3.2 Verify Token (Get Current User)

GET

`/api/auth/me`

Retrieve authenticated user profile

Requires Bearer Token.

Response Example

Postman interface showing a GET request to `http://localhost:5000/api/auth/me`. The request has a Bearer Token in the Authorization header. The response is a 200 OK status with a JSON body:

```
2 {
3   "success": true,
4   "message": "Success",
5   "data": {
6     "user_id": 2,
7     "username": "soc_manager",
8     "email": "manager@soc.com",
9     "role": "soc_manager",
10    "created_at": "2026-04-28 06:03:30.3100000",
11    "last_login": "2026-05-08 21:21:00.5166667",
12    "is_active": true
13  }
14 }
```


4. Alerts

Manage security alerts in the SIEM. All endpoints require Bearer Token authentication.

4.1 Create Alert

POST

/api/alerts

Create a new security alert

Request Example

The screenshot shows a REST client interface with a workspace named 'My Workspace'. On the left, a 'COLLECTIONS' sidebar lists several requests, with 'GET alerts' selected. The main panel displays a 'POST' request to 'http://localhost:5000/api/alerts'. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type and a masked token. Below this, a note states: 'The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.' The 'Body' tab shows a JSON response with a status of '201 Created', a response time of '24 ms', and a size of '1.33 KB'. The JSON data is as follows:

```

{
  "data": {
    "alert_id": 100,
    "source_ip": null,
    "dest_ip": null,
    "severity": "low",
    "alert_type": "Manual",
    "timestamp": "2026-05-08 21:41:11.453333",
    "status": "new",
    "description": null,
    "threat_intel_data": null,
    "mitre_stage": null,
    "confidence score": 100
  }
}

```

At the bottom of the interface, there are tabs for 'Connect Git', 'Console', and 'Terminal', and a footer with 'Globals', 'Vault', and 'Tools'.

4.2 Get Alert Statistics

GET

/api/alerts/stats

Retrieve aggregated alert statistics

Request Example

The screenshot shows a REST client interface with a collection named 'alerts'. A GET request is configured for the endpoint `http://localhost:5000/api/alerts/stats`. The authentication is set to 'Bearer Token'. The response is a 200 OK status with a 45 ms response time and 1.73 KB of data. The response body is a JSON object:

```

{
  "success": true,
  "message": "Success",
  "data": {
    "by_severity": [
      {
        "severity": "critical",
        "count": 13
      },
      {
        "severity": "high",
        "count": 16
      },
      {
        "severity": "low",
        "count": 30
      },
      {
        "severity": "medium",
        "count": 21
      }
    ]
  }
}

```

4.3 Create Alert

POST

`/api/alerts`

Create a new security alert

Request Body

Field	Type	Required	Description
<code>title</code>	string	Required	Alert title
<code>severity</code>	string	Required	Severity level: critical High medium low
<code>source_ip</code>	string	Optional	Source IP address
<code>description</code>	string	Optional	Detailed description
<code>alert_type</code>	string	Optional	Type: Brute Force Manual etc.

Request Example

GET New Request POST login_request LogEase_Local GET verify_request POST alerts LogEase_Local

New Collection > alerts

POST http://localhost:5000/api/alerts

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```

1 {
2   "title": "Suspicious Login Attempt",
3   "severity": "High",
4   "source_ip": "45.33.32.156",
5   "description": "Multiple failed login attempts detected",
6   "alert_type": "Brute Force"
7 }

```

Body Cookies Headers 25 Test Results 201 Created • 19 ms • 1.38 KB Save Response

JSON Preview Visualize

```

1 {
2   "success": true,
3   "message": "Alert created",
4   "data": {
5     "alert_id": 102,
6     "source_ip": "45.33.32.156",
7     "dest_ip": null,
8     "severity": "High",
9     "alert_type": "Brute Force",
10    "timestamp": "2026-05-08 21:45:59.833333",
11    "status": "new",
12    "description": "Multiple failed login attempts detected",
13    "threat_intel_data": null,
14    "mitre_stage": null,
15    "confidence_score": 100
16  }
17 }

```

Connect Git Console Terminal Globals Vault Tools

4.4 Acknowledge Alert

PUT

/api/alerts/{alert_id}/acknowledge

Mark an alert as acknowledged

Path Parameters

Parameter	Type	Required	Description
alert_id	integer	Required	Alert ID to acknowledge

Request Example

GET New Request POST login_request LogEase_Local GET verify_request PUT alerts LogEase_Local

New Collection > alerts

PUT http://localhost:5000/api/alerts/102/acknowledge

Docs Params Authorization Headers 9 Body Scripts Settings

Auth Type Bearer Token Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 25 Test Results 200 OK • 37 ms • 2.5 KB Save Response

JSON Preview Visualize

```

1 {
2   "success": true,
3   "message": "Alert acknowledged",
4   "data": {
5     "alert_id": 102,
6     "source_ip": "45.33.32.156",
7     "dest_ip": null,
8     "severity": "high",
9     "alert_type": "Brute Force",
10    "timestamp": "2026-05-08 21:45:59.833333",
11    "status": "acknowledged",
12    "description": "Multiple failed login attempts detected",
13    "threat_intel_data": {
14      "abuseipdb": {
15        "ip": "45.33.32.156",
16        "abuseScore": 7,
17        "totalReports": 2,
18        "country": "US",
19        "isp": "Linode",
20        "domain": "linode.com",
21        "usageType": "Data Center/Web Hosting/Transit",
22        "isWhitelisted": false,
23        "lastReportedAt": "2026-04-26T13:19:54+00:00",
24        "risk_level": "Low",
25        "virustotal": {
26          "ip": "45.33.32.156",
27          "malicious": 4,
28          "suspicious": 8,
29          "harmless": 55,
30          "undetected": 33,
31          "total_engines": 92
32        }
33      }
34    }
35  }
36 }

```

Connect Git Console Terminal Globals Vault Tools

5. Incidents

Create and manage security incidents. All endpoints require Bearer Token authentication.

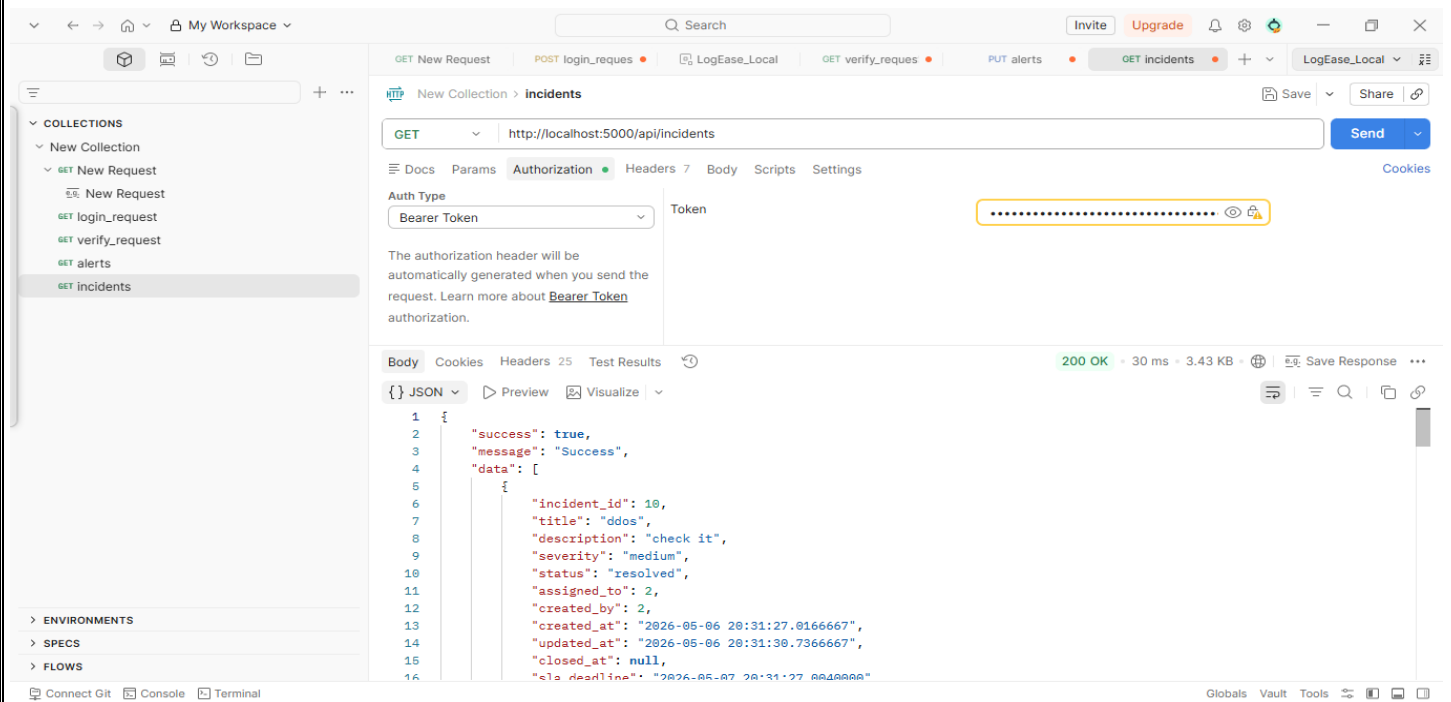
5.1 Get All Incidents

GET

/api/incidents

Retrieve list of all incidents

Request Example



5.2 Create Incident

POST

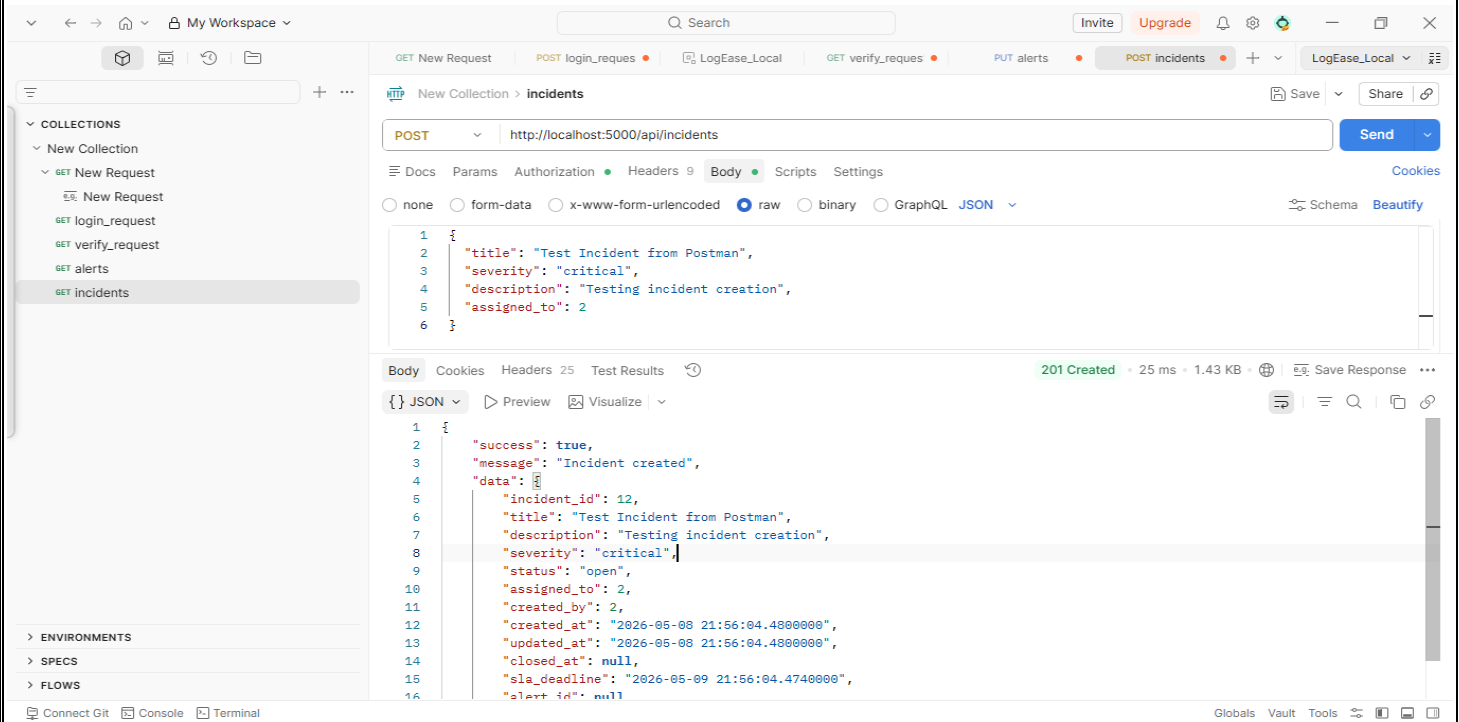
/api/incidents

Create a new security incident

Request Body

Field	Type	Required	Description
title	string	Required	Incident title
severity	string	Required	critical high medium low
description	string	Required	Incident description
assigned_to	integer	Required	User ID of assignee

Request Example



5.3 Update Incident

PUT

/api/incidents/{incident_id}

Update an existing incident

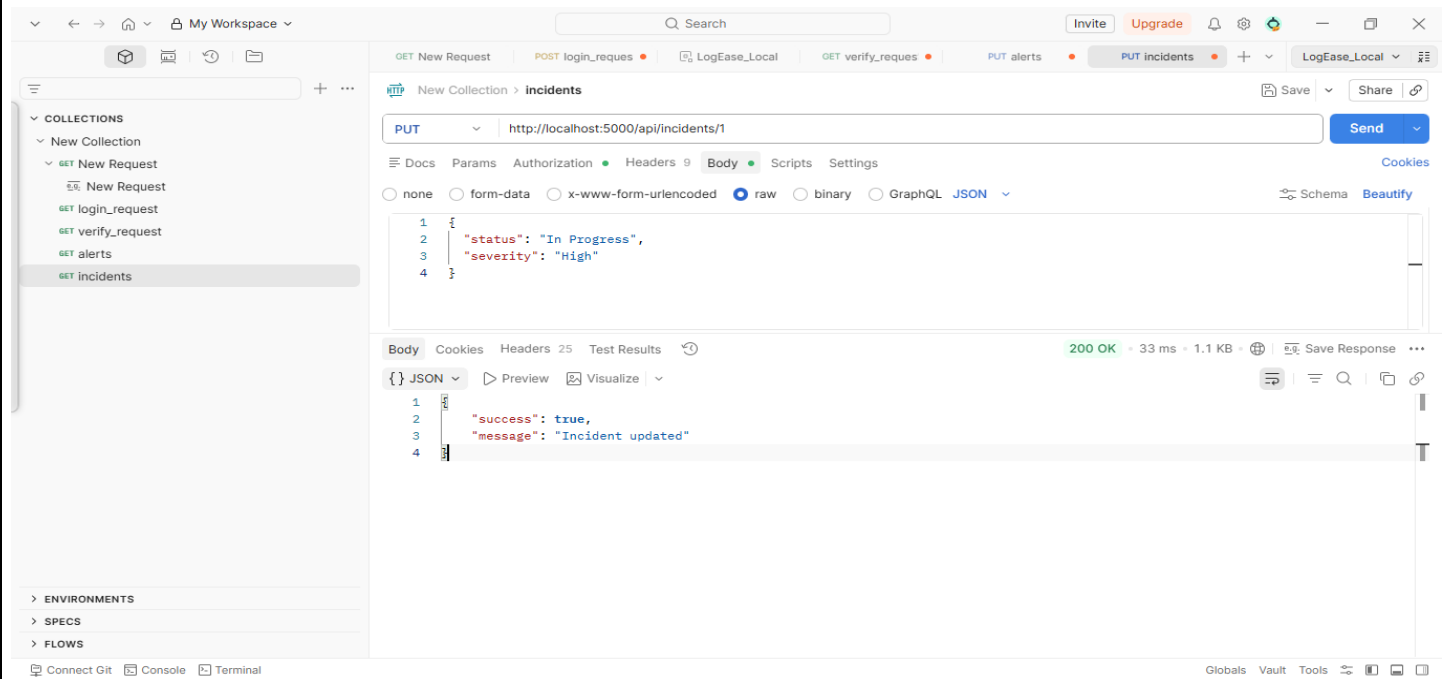
Path Parameters

Parameter	Type	Required	Description
incident_id	integer	Required	Incident ID to update

Request Body

Field	Type	Required	Description
status	string	Optional	open In Progress resolved closed
severity	string	Optional	critical High medium low

Request Example



6. Threat Intelligence

Scan IOCs, retrieve recent threat data, and block malicious entities. All endpoints require Bearer Token authentication.

6.1 Scan IOC

POST	/api/threat-intel/scan	Scan an Indicator of Compromise (IOC)
-------------	------------------------	---------------------------------------

Request Body

Field	Type	Required	Description
ioc_value	string	Required	The IP address or URL to scan
ioc_type	string	Required	Type: ip url domain hash

Request Example

GET New Request POST login_requ LogEase_Local GET verify_requ PUT alerts PUT incidents POST Threat_int LogEase_Local

New Collection > Threat_intel

POST http://localhost:5000/api/threat-intel/scan

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "ioc_value": "45.33.32.156",
3   "ioc_type": "ip"
4 }
```

Body Cookies Headers 25 Test Results

200 OK - 17 ms - 1.6 KB

```
2 {
3   "success": true,
4   "message": "Success",
5   "data": {
6     "ip": "45.33.32.156",
7     "continent_name": "North America",
8     "country_name": "United States",
9     "country_code2": "US",
10    "state_prov": "California",
11    "city": "Fremont",
12    "zipcode": "94538",
13    "latitude": 37.65148,
14    "longitude": -121.9833,
15    "isp": "Linode",
16    "organization": "Akamai Technologies, Inc.",
17    "timezone": "America/Los_Angeles",
18  }
19 }
```

6.2 Get Recent Threat Intelligence

GET`/api/threat-intel/recent`*Retrieve recently observed threat IOCs*

Request Example

GET New Request POST login_requ LogEase_Local GET verify_requ PUT alerts PUT incidents GET Threat_int LogEase_Local

New Collection > Threat_intel

GET http://localhost:5000/api/threat-intel/recent

Docs Params Authorization Headers 7 Body Scripts Settings

Auth Type: Bearer Token

Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 25 Test Results

200 OK - 19 ms - 2.4 KB

```
5 [
6   {
7     "ioc_value": "45.33.32.156",
8     "ioc_type": "ip",
9     "threat_score": 0,
10    "malicious": 0,
11    "hazmlless": 0,
12    "last_seen": "2026-05-08 21:46:02.3466667"
13  },
14  {
15    "ioc_value": "https://igetintopc.com/",
16    "ioc_type": "url",
17    "threat_score": 13,
18    "malicious": 12,
19    "hazmlless": 49,
20    "last_seen": "2026-05-07 20:09:10.7066667"
21  }
22 ]
```

6.3 Block IP / Entity

POST`/api/actions/block`*Block a malicious IP or entity*

Request Body

Field	Type	Required	Description
<code>entity_value</code>	string	Required	IP address or entity to block
<code>entity_type</code>	string	Required	Type: ip domain url
<code>reason</code>	string	Required	Reason for blocking

Request Example

The screenshot shows a REST client interface with a workspace named 'My Workspace'. A collection named 'Threat_intel' is selected, containing a 'POST' request to 'http://localhost:5000/api/actions/block'. The request body is raw JSON:

```
1 {
2   "entity_value": "45.33.32.156",
3   "entity_type": "ip",
4   "reason": "Confirmed malicious - VirusTotal 45/72"
5 }
```

The response is a 201 Created status, 26 ms, 1.32 KB. The response body is JSON:

```
1 {
2   "success": true,
3   "message": "IP blocked successfully",
4   "data": {
5     "block_id": 11,
6     "entity_type": "ip",
7     "entity_value": "45.33.32.156",
8     "reason": "Confirmed malicious - VirusTotal 45/72",
9     "blocked_by": 2,
10    "blocked_at": "2026-05-08 22:08:19.1733333",
11    "expires_at": null,
12    "status": "active"
13  }
14 }
```

7. Geo-Location Lookup

Look up geographic and ISP information for IP addresses. All endpoints require Bearer Token authentication.

7.1 Single IP Lookup

GET`/api/geo/{ip}`

Get geo-location data for a single IP address

Path Parameters

Parameter	Type	Required	Description
ip	string	Required	IP address to look up (e.g. 8.8.8.8)

Request Example

The screenshot shows a REST client interface with a collection named 'Threat_intel'. A GET request is configured with the URL `http://localhost:5000/api/geo/8.8.8.8` and 'Bearer Token' authorization. The response is a 200 OK status with a 758 ms response time and 1.52 KB of data. The response body is a JSON object containing geo-location data for the IP address 8.8.8.8.

```

{
  "ip": "8.8.8.8",
  "continent_name": "North America",
  "country_name": "United States",
  "country_code2": "US",
  "state_prov": "California",
  "city": "Mountain View",
  "zipcode": "94043-1351",
  "latitude": 37.4224,
  "longitude": -122.08421,
  "isp": "Google LLC",
  "organization": "Google LLC",
  "timezone": "America/Los_Angeles",
  "currency": "US Dollar",
  "connection_type": "",
  "threat": {}
}

```

7.2 Bulk IP Lookup

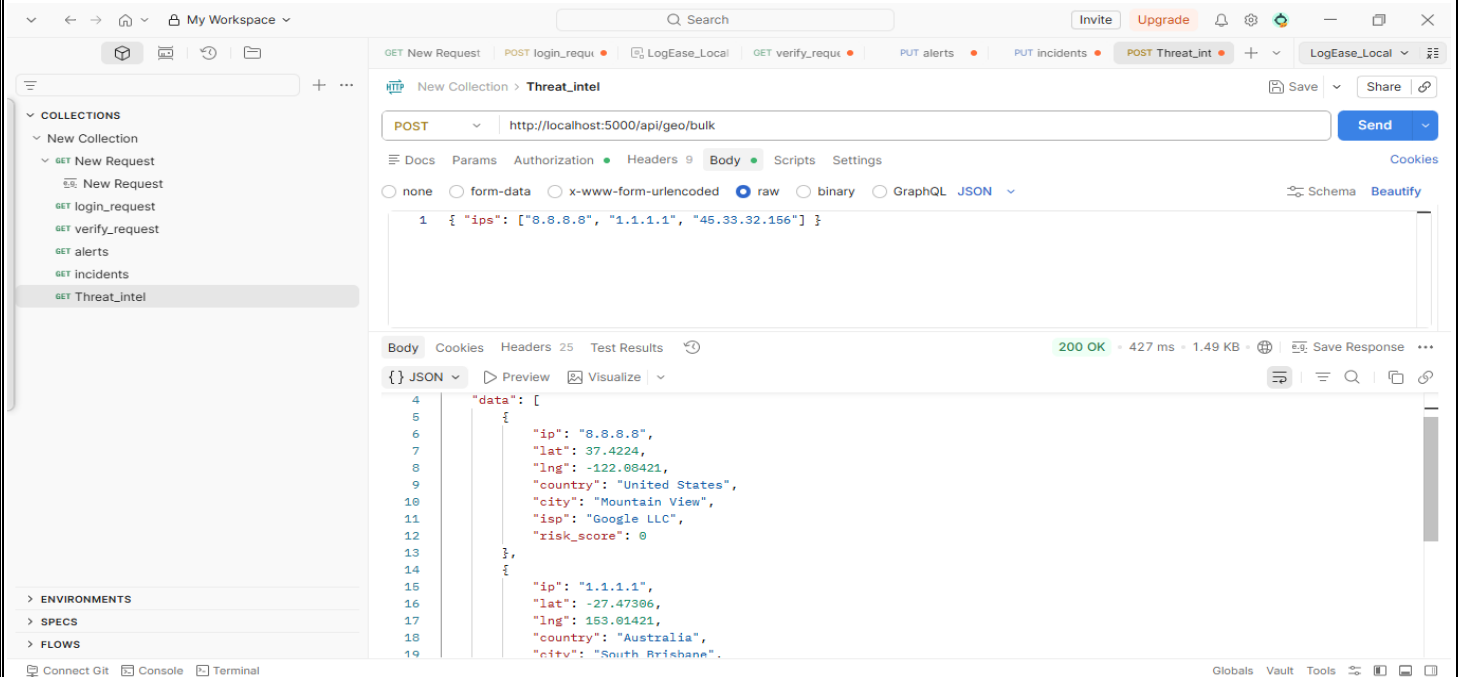
POST`/api/geo/bulk`

Get geo-location data for multiple IPs at once

Request Body

Field	Type	Required	Description
ips	array	Required	Array of IP address strings

Request Example



LogEase API client interface showing a POST request to `http://localhost:5000/api/geo/bulk`. The request body is a JSON array of IP addresses: `[{"ips": ["8.8.8.8", "1.1.1.1", "45.33.32.156"]}]`. The response is a 200 OK status with a 427 ms response time and 1.49 KB of data. The response body is a JSON object containing an array of location data for the provided IP addresses.

```

{
  "data": [
    {
      "ip": "8.8.8.8",
      "lat": 37.4224,
      "lng": -122.08421,
      "country": "United States",
      "city": "Mountain View",
      "isp": "Google LLC",
      "risk_score": 0
    },
    {
      "ip": "1.1.1.1",
      "lat": -27.47306,
      "lng": 153.01421,
      "country": "Australia",
      "city": "South Brisbane"
    }
  ]
}

```

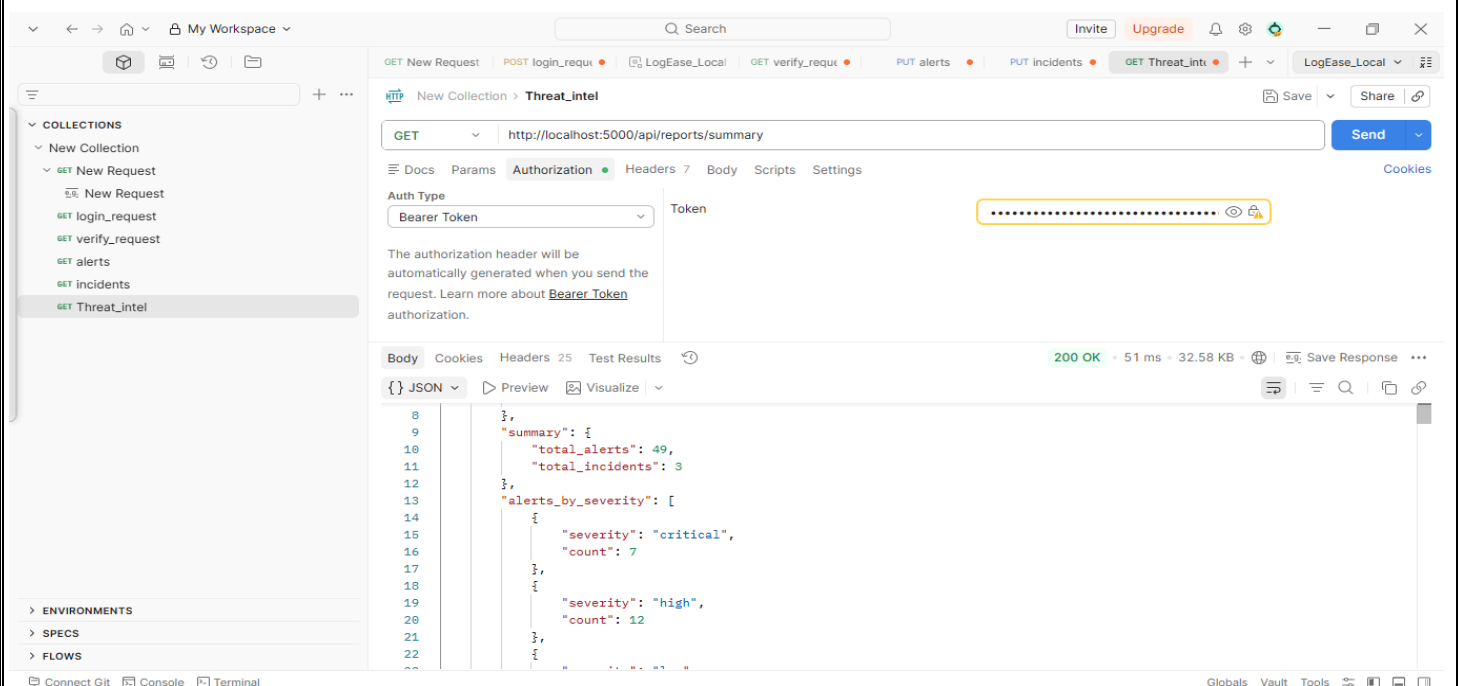
8. Reports

Generate and download security reports in multiple formats. All endpoints require Bearer Token authentication.

8.1 Report Summary (JSON)

GET	<code>/api/reports/summary</code>	Get a full JSON summary of alerts and incidents
------------	-----------------------------------	---

Request Example



LogEase API client interface showing a GET request to `http://localhost:5000/api/reports/summary`. The request is configured with Bearer Token authentication. The response is a 200 OK status with a 51 ms response time and 32.58 KB of data. The response body is a JSON object containing a summary of alerts and incidents, including total counts and a breakdown by severity.

```

{
  "summary": {
    "total_alerts": 49,
    "total_incidents": 3
  },
  "alerts_by_severity": [
    {
      "severity": "critical",
      "count": 7
    },
    {
      "severity": "high",
      "count": 12
    }
  ]
}

```

8.2 Alert Report — PDF

GET

/api/reports/alerts/pdf

Download a formatted PDF report of all alerts

Response Headers

Content-Type

application/pdf

Status

200 OK — binary PDF stream

Request Example

The screenshot shows a REST client interface with a sidebar on the left containing a 'COLLECTIONS' list. The main panel displays a GET request to `http://localhost:5000/api/reports/alerts/pdf`. The 'Authorization' tab is active, showing a 'Bearer Token' field. The response preview shows a PDF report dashboard with the following metrics:

Metric	Value
TOTAL EVENTS	49
CRITICAL THREATS	7
HIGH ALERTS	12
MEDIUM ALERTS	13
LOW ALERTS	17
ACKNOWLEDGED	3
RESOLVED EVENTS	0
UNIQUE SOURCE IPS	47

Below the metrics is a section titled '02. ALERT SEVERITY BREAKDOWN' with a horizontal bar chart showing the distribution of alerts by severity: CRITICAL (7), HIGH (12), LOW (17), and MEDIUM (13).

The PDF report includes an Executive Summary dashboard with metrics: Total Events, Critical Threats, High/Medium/Low Alerts, Acknowledged count, Resolved Events, and Unique Source IPs. It also contains an Alert Severity Breakdown chart.

8.3 Alert Report — CSV

GET

/api/reports/alerts/csv?type=alerts&format=csv

Download a CSV export of all alert data

Query Parameters

Parameter	Type	Required	Description
type	string	Required	Report type: alerts
format	string	Required	Output format: csv

CSV Report Structure

Report Title	SOC Security Alerts Report
Classification	INTERNAL CONFIDENTIAL
Generated By	LogEase SIEM v2.0 — Enterprise Edition
Sections	Executive Summary, Alert Details, Metric breakdowns

Request Example

The screenshot shows a REST client interface with the following details:

- Request Method:** GET
- URL:** `http://localhost:5000/api/reports/alerts/csv?type=alerts&format=csv`
- Authorization:** Bearer Token
- Response Status:** 200 OK
- Response Time:** 76 ms
- Response Size:** 10.29 KB
- Response Body (Raw):**

```

1 LOGEASE SIEM - SECURITY OPERATIONS REPORT,
2 -----,
3 Report Title,SOC Security Alerts Report
4 Report Date,"09/05/2026, 3:21:48 am"
5 Period From,"May 1, 2026"
6 Period To,"May 9, 2026"
7 Issuing Analyst,soc_manager
8 Classification,INTERNAL CONFIDENTIAL
9 Generated By,LogEase SIEM v2.0 - Enterprise Edition
10 -----,
11
12 EXECUTIVE SUMMARY,
13 -----,
14 Metric,Value,Status
15 TOTAL EVENTS,49,
16 CRITICAL THREATS 7

```

9. Dashboard

Dashboard endpoints provide summary statistics and recent activity. Both soc manager and Viewer roles have access. All endpoints require Bearer Token authentication.

9.1 Dashboard Summary

GET	<code>/api/dashboard/summary</code>	<i>Get high-level security metrics</i>
------------	-------------------------------------	--

Request Example

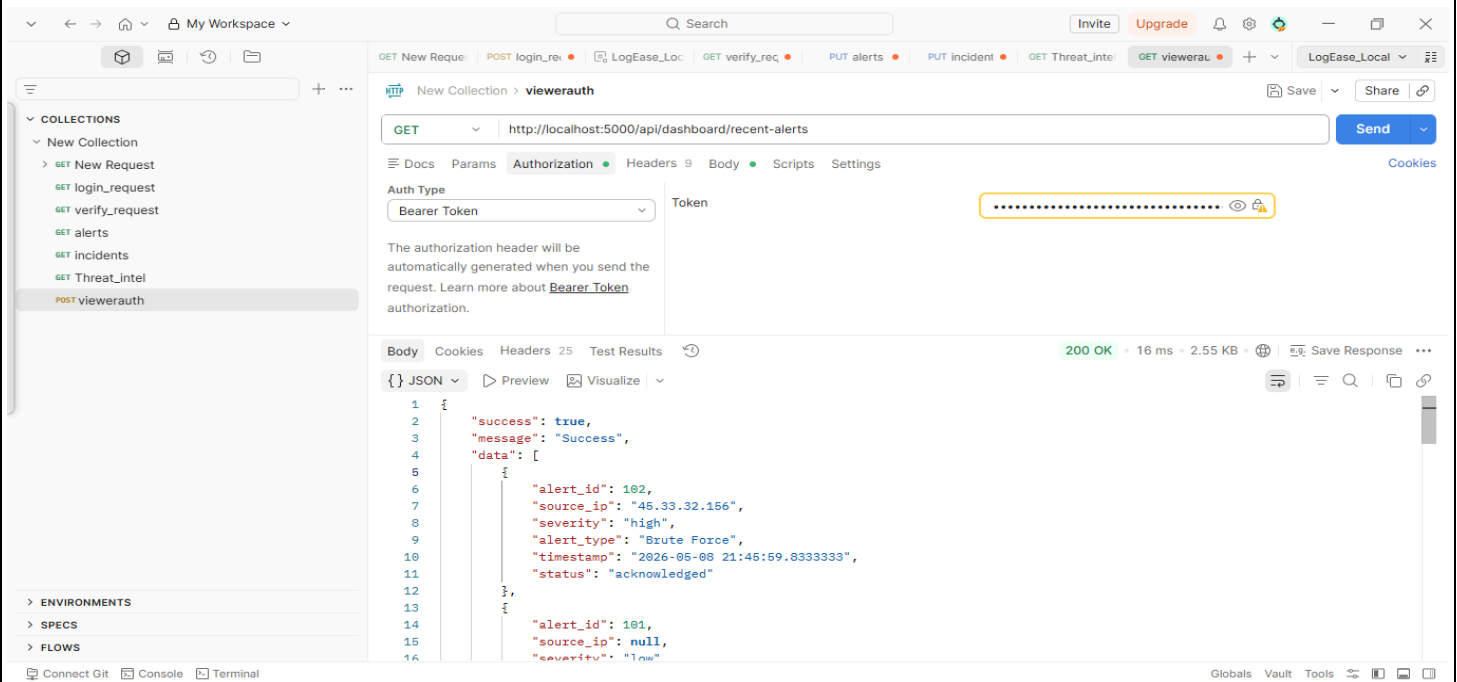
The screenshot shows a REST client interface with a collection named 'viewerauth'. A GET request is configured to 'http://localhost:5000/api/dashboard/summary'. The authorization is set to 'Bearer Token'. The response is a 200 OK status with a 23 ms response time and 1.18 KB of data. The response body is a JSON object:

```
{
  "success": true,
  "message": "Success",
  "data": {
    "critical_threats": 11,
    "active_incidents": 4,
    "blocked_attacks": 0,
    "system_health": 7
  }
}
```

9.2 Recent Alerts

GET	<code>/api/dashboard/recent-alerts</code>	<i>Get the most recent security alerts</i>
------------	---	--

Request Example



10. User Management

Endpoints for managing platform users. These endpoints are restricted to soc_manager role only. Viewer role receives 403 Forbidden on all user management operations.

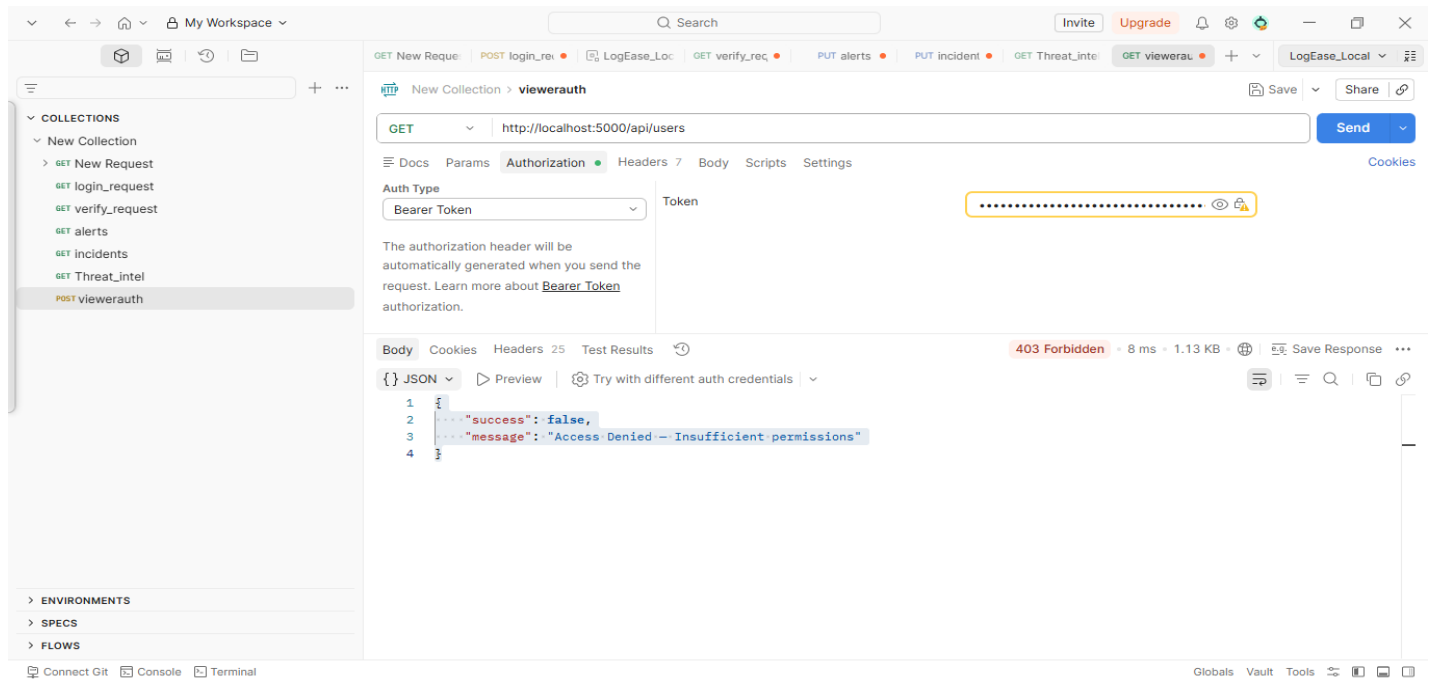
10.1 Get All Users

GET	/api/users	Retrieve list of all platform users
------------	------------	-------------------------------------

Access Control

Role	Response
soc_manager	200 OK — returns user list
Viewer	403 Forbidden — insufficient permissions
Analyst	403 Forbidden — insufficient permissions

Request Example



10.2 Create User

POST	/api/users	Create a new platform user (admin only)
-------------	------------	---

Request Body

Field	Type	Required	Description
email	string	Required	New user's email address
password	string	Required	New user's password
role	string	Required	Role: soc_manager Viewer Analyst

Request Example

The screenshot shows a REST client interface with a collection named 'viewerauth'. A POST request is configured to 'http://localhost:5000/api/users'. The request body is a JSON object:

```
{  "email": "new@soc.com",  "password": "New123!",  "role": "Analyst"}
```

. The response status is '403 Forbidden' with a message:

```
{  "success": false,  "message": "Access Denied - Insufficient permissions"}
```

. The left sidebar shows a list of collections, and the bottom status bar indicates 'Connect Git', 'Console', and 'Terminal'.

10.3 Register Viewer

POST`/api/auth/register`*Register as Viewer*

Request Example

The screenshot shows a REST client interface with a collection named 'viewerauth'. A POST request is configured to 'http://localhost:5000/api/auth/register'. The request body is a JSON object:

```
{  "username": "test_viewer",  "email": "testviewer@soc.com",  "role": "Viewer",  "password": "Viewer@123",  "confirmPassword": "Viewer@123"}
```

. The response status is '201 Created' with a message:

```
{  "success": true,  "message": "User registered successfully",  "data": {    "user_id": 8  }}
```

. The left sidebar shows a list of collections, and the bottom status bar indicates 'Connect Git', 'Console', and 'Terminal'.

10.4 Login Viewer

POST`/api/auth/Login`*Login as Viewer*

Request Example

The screenshot shows the Postman interface with a new collection named 'viewerauth'. The selected request is a POST to 'http://localhost:5000/api/auth/register'. The body is raw JSON with the following content:

```
1 {
2   "username": "test_viewer",
3   "email": "testviewer@soc.com",
4   "role": "Viewer",
5   "password": "Viewer@123",
6   "confirmPassword": "Viewer@123"
7 }
```

The response is a 201 Created status with a 157 ms response time and 1.13 KB body. The response body is JSON:

```
1 {
2   "success": true,
3   "message": "User registered successfully",
4   "data": {
5     "user_id": 8
6   }
7 }
```

10.5 Incidents Access by Viewer

POST`/api/incidents`*Incidents access by Viewer*

Request Example

The screenshot shows the Postman interface with the same 'viewerauth' collection. The selected request is a POST to 'http://localhost:5000/api/incidents'. The 'Authorization' tab is active, showing 'Bearer Token' as the auth type. The token field is masked with dots. Below the token field, a message states: 'The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.'

The response is a 403 Forbidden status with a 5 ms response time and 1.13 KB body. The response body is JSON:

```
1 {
2   "success": false,
3   "message": "Access Denied -- Insufficient permissions"
4 }
```

10.6 Get Users by Viewer

GET`/api/users`*Get Users by Viewer*

Request Example

The screenshot shows the Postman interface with a new collection named 'viewerauth'. The request is a GET to 'http://localhost:5000/api/users' using 'Bearer Token' authorization. The response is a 403 Forbidden error with a message: 'Access Denied - Insufficient permissions'.

```
{
  "success": false,
  "message": "Access Denied - Insufficient permissions"
}
```

10.7 Delete Incidents by Viewer

DELETE`/api/incidents/10`*Delete Incidents by Viewer*

Request Example

The screenshot shows the Postman interface with a new collection named 'viewerauth'. The request is a DELETE to 'http://localhost:5000/api/incidents/10' using 'Bearer Token' authorization. The response is a 403 Forbidden error with a message: 'Access Denied - Insufficient permissions'.

```
{
  "success": false,
  "message": "Access Denied - Insufficient permissions"
}
```

11. Viewer Role — Access Restrictions

The Viewer role is read-only. The following table documents which operations are permitted and which return 403 Forbidden when accessed with a Viewer token.

Endpoint	Role	Status	Notes
GET /api/alerts	Viewer	200 OK	Can view all alerts with threat intel data
GET /api/incidents	Viewer	200 OK	Can view all incidents
GET /api/dashboard/summary	Viewer	200 OK	Can view dashboard metrics
GET /api/dashboard/recent-alerts	Viewer	200 OK	Can view recent alerts
GET /api/auth/me	Viewer	200 OK	Can view own profile
POST /api/incidents	Viewer	403 Forbidden	Cannot create incidents
PUT /api/incidents/{id}	Viewer	403 Forbidden	Cannot update incidents
DELETE /api/incidents/{id}	Viewer	403 Forbidden	Cannot delete incidents
POST /api/alerts	Viewer	403 Forbidden	Cannot create alerts
GET /api/users	Viewer	403 Forbidden	Cannot list users
POST /api/users	Viewer	403 Forbidden	Cannot create users
POST /api/actions/block	Viewer	403 Forbidden	Cannot block IPs
POST /api/threat-intel/scan	Viewer	403 Forbidden	Cannot trigger scans

12. Analyst Role Login

POST

/api/auth/login

Login access by Analyst

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A POST request is configured to 'http://localhost:5000/api/auth/login'. The request body is a JSON object with 'email' and 'password' fields. The response is a 200 OK status with a JSON body containing 'success', 'message', and 'data' fields. The 'data' field includes a 'token', a 'user' object with 'user_id', 'username', 'email', and 'role'.

```

1 {
2   "email": "analyst@soc.com",
3   "password": "Analyst123!"
4 }

```

```

1 {
2   "success": true,
3   "message": "Login successful",
4   "data": {
5     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJ1c2VybmFtZSI6ImFuYXV5bW50c3QxIiwiaW1haWwiOiJhbmFseXN0QHNvYy5jb20iLCJyb2x1Ijo1YW5hbH1zdCI6Im1hdCI6MTc3ODY9ODU4MywiZmxhIjozNjc3MzQzZjQ.IxTwhH_M0_48NoQyW1u21yY3z4CBzNlBzpFvZZIo8Eg",
6     "user": {
7       "user_id": 3,
8       "username": "analyst1",
9       "email": "analyst@soc.com",
10      "role": "analyst"
11     }
12   }
13 }

```

12.1 Analyst Get Alerts

GET

/api/alerts

Analyst Get Alerts

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A GET request is configured to 'http://localhost:5000/api/alerts' with Bearer Token authentication. The response is a 200 OK status with a JSON body containing two alert objects.

```

{
  "alert_id": 184,
  "source_ip": null,
  "dest_ip": null,
  "severity": "low",
  "alert_type": "Manual",
  "timestamp": "2026-05-13 05:04:14.4100000",
  "status": "new",
  "description": null,
  "threat_intel_data": null,
  "mitre_stage": null,
  "confidence_score": 100
},
{
  "alert_id": 183,
  "source_ip": "45.33.32.23",
  "dest_ip": null,
  "severity": "critical",
  "alert_type": "Manual",
  "timestamp": "2026-05-13 05:04:14.4100000",
  "status": "new",
  "description": null,
  "threat_intel_data": null,
  "mitre_stage": null,
  "confidence_score": 100
}

```

12.2 Analyst Get Incidents

GET	/api/incidents	Analyst Get Incidents
-----	----------------	-----------------------

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A GET request is configured to 'http://localhost:5000/api/incidents' with Bearer Token authentication. The response is a 200 OK status with a JSON body containing an incident object.

```

{
  "description": "Testing incident creation",
  "severity": "critical",
  "status": "open",
  "assigned_to": 2,
  "created_by": 2,
  "created_at": "2026-05-08 21:56:04.4800000",
  "updated_at": "2026-05-08 21:56:04.4800000",
  "closed_at": null,
  "sla_deadline": "2026-05-09 21:56:04.4740000",
  "alert_id": null,
  "assigned_username": "soc_manager",
  "created_by_username": "soc_manager"
},
{
  "incident_id": 10,
  "description": "Testing incident creation",
  "severity": "critical",
  "status": "open",
  "assigned_to": 2,
  "created_by": 2,
  "created_at": "2026-05-08 21:56:04.4800000",
  "updated_at": "2026-05-08 21:56:04.4800000",
  "closed_at": null,
  "sla_deadline": "2026-05-09 21:56:04.4740000",
  "alert_id": null,
  "assigned_username": "soc_manager",
  "created_by_username": "soc_manager"
}

```

12.3 Analyst Add Incident

POST`/api/incidents`*Analyst Add Incident*

Request Example

The screenshot displays a REST client interface with a sidebar on the left showing a collection of requests. The main panel shows a POST request to `http://localhost:5000/api/incidents` with a raw JSON body. The response is a 201 status code, indicating the incident was created successfully. The response body contains details about the incident, including its ID, title, severity, description, status, assigned user, and timestamps.

```
1 {
2   "title": "Brute Force Attack Detected",
3   "severity": "high",
4   "description": "Multiple failed login attempts from IP 192.168.1.100",
5   "assigned_to": 2
6 }
```

```
1 {
2   "success": true,
3   "message": "Incident created",
4   "data": {
5     "incident_id": 13,
6     "title": "Brute Force Attack Detected",
7     "description": "Multiple failed login attempts from IP 192.168.1.100",
8     "severity": "high",
9     "status": "open",
10    "assigned_to": 2,
11    "created_by": 3,
12    "created_at": "2026-05-13 05:07:24.9000000",
13    "updated_at": "2026-05-13 05:07:24.9000000",
14    "closed_at": null,
15    "sla_deadline": "2026-05-14 05:07:24.8960000",
16    "alert_id": null
17  }
18 }
```

12.4 Analyst Update Incident

PUT`/api/incidents/13`*Analyst Update Incident*

Request Example

My Workspace

Search

Invite Upgrade

LogEase_Local

PUT analyst

PUT http://localhost:5000/api/incidents/13

Send

Docs Params Authorization Headers 9 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "status": "resolved",
3   "severity": "high"
4 }
```

Schema Beautify

Body Cookies Headers 25 Test Results

200 OK • 32 ms • 1.51 KB

Save Response

JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Incident updated",
4   "data": {
5     "incident_id": 13,
6     "title": "Brute Force Attack Detected",
7     "description": "Multiple failed login attempts from IP 192.168.1.100",
8     "severity": "high",
9     "status": "resolved",
10    "assigned_to": 2,
11    "created_by": 3,
12    "created_at": "2026-05-13 05:07:24.900000",
13    "updated_at": "2026-05-13 05:21:39.536667",
14    "closed_at": "2026-05-13 05:21:39.536667",
15    "sla_deadline": "2026-05-14 05:07:24.896000",
16    "alert_id": null
17  }
18 }
```

Connect Git Console Terminal

Globals Vault Tools

12.5 Analyst Get Summary

GET	/api/reports/summary	Analyst Get Summary Report
-----	----------------------	----------------------------

Request Example

My Workspace

Search

Invite Upgrade

LogEase_Local

GET analyst

GET http://localhost:5000/api/reports/summary

Send

Docs Params Authorization Headers 9 Body Scripts Settings

Auth Type: Bearer Token

Token: [REDACTED]

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 25 Test Results

200 OK • 93 ms • 32.83 KB

Save Response

JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Success",
4   "data": {
5     "report_period": {
6       "from": "2026-05-06T00:00:00.000Z",
7       "to": "2026-05-13T23:59:59.999Z"
8     },
9     "summary": {
10      "total_alerts": 131,
11      "total_incidents": 4
12    },
13    "alerts_by_severity": [
14      {
15        "severity": "critical",
16        "count": 7
17      }
18    ]
19  }
20 }
```

Connect Git Console Terminal

Globals Vault Tools

12.6 Analyst Delete Incident

DELETE

/api/incidents/13

Analyst Delete Incident

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. The selected request is a DELETE request to the endpoint `http://localhost:5000/api/incidents/13`. The request body is a JSON object:

```
1 {
2   "title": "Test Alert from Analyst",
3   "severity": "high",
4   "description": "This should be blocked"
5 }
```

The response is a 403 Forbidden status with a message: "Access Denied - Insufficient permissions". The response body is shown in JSON format:

```
1 {
2   "success": false,
3   "message": "Access Denied - Insufficient permissions"
4 }
```

12.7 Analyst Get Users

GET

/api/users

Analyst Get Incident

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A GET request is configured to 'http://localhost:5000/api/users' with 'Bearer Token' authorization. The response is a 403 Forbidden status, 10 ms, 1.13 KB. The JSON body is:

```
{
  "success": false,
  "message": "Access Denied - Insufficient permissions"
}
```

12.8 Analyst Block IP

POST`/api/actions/block`*Analyst Block Ip Addresses*

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A POST request is configured to 'http://localhost:5000/api/actions/block' with 'raw' body type. The request body is:

```
{
  "entity_value": "192.168.1.100",
  "entity_type": "ip",
  "reason": "Malicious activity detected"
}
```

The response is a 201 Created status, 33 ms, 1.31 KB. The JSON body is:

```
{
  "success": true,
  "message": "IP blocked successfully",
  "data": {
    "block_id": 12,
    "entity_type": "ip",
    "entity_value": "192.168.1.100",
    "reason": "Malicious activity detected",
    "blocked_by": 3,
    "blocked_at": "2026-05-13 06:09:28.8733333",
    "expires_at": null,
    "status": "active"
  }
}
```

13 Error Handling

Entered Invalid Token

Request Example

The screenshot shows the Postman interface with a new collection named 'analyst'. A GET request is configured to 'http://localhost:5000/api/alerts' with 'Bearer Token' authorization. The token 'xTwhH_M0_4BNoQyW1u21yY3z4CBeg' is entered. The response is a 401 Unauthorized status with a 11 ms response time and 1.1 KB body. The JSON body contains:

```
{  "success": false,  "message": "Invalid token"}
```

13.1 No Token Provided

When Not Issues Token

Request Example

The screenshot shows the Postman interface with the same collection 'analyst'. The GET request is configured to 'http://localhost:5000/api/alerts' with 'No Auth' authorization. The response is a 401 Unauthorized status with a 7 ms response time and 1.1 KB body. The JSON body contains:

```
{  "success": false,  "message": "No token provided"}
```

13.2 Invalid Id

Entered Invalid Id

Request Example

The screenshot shows the Postman interface. On the left, a collection named 'analyst' is expanded, showing a 'POST analyst' request. The main panel displays a 'GET' request to 'http://localhost:5000/api/alerts/999999'. The 'Authorization' tab is selected, showing 'Bearer Token' as the auth type and a masked token. The 'Body' tab shows a '404 Not Found' status with a response time of 15 ms and a body of:

```
{  \"success\": false,  \"message\": \"Alert not found\"}
```

13.3 Login with Invalid Credentials

Login with Invalid credentials

Request Example

The screenshot shows the Postman interface. On the left, a collection named 'analyst' is expanded, showing a 'POST analyst' request. The main panel displays a 'POST' request to 'http://localhost:5000/api/auth/login'. The 'Body' tab is selected, showing 'raw' as the body type and a JSON body:

```
{  \"email\": \"not-an-email\",  \"password\": \"Test@123\"}
```

 The 'Body' tab shows a '401 Unauthorized' status with a response time of 21 ms and a body of:

```
{  \"success\": false,  \"message\": \"Invalid credentials\"}
```

13.4 Wrong Password Attempts

Invalid or Wrong Password

Request Example

The screenshot shows the Postman interface with a new collection named 'analyst'. A POST request is configured to 'http://localhost:5000/api/auth/login'. The body is set to 'raw' and contains the following JSON:

```
1 {
2   "email": "manager@soc.com",
3   "password": "wrongpassword123"
4 }
```

The response is a 401 Unauthorized status with a 103 ms response time and 1.11 KB of data. The response body is JSON:

```
1 {
2   "success": false,
3   "message": "Invalid credentials"
4 }
```

13.5 SQL Injection

Try to get alert title by parameters id

Request Example

The screenshot shows the Postman interface with a new collection named 'analyst'. A GET request is configured to 'http://localhost:5000/api/alerts?title=' OR '1'=1'. The 'Authorization' tab is selected, showing 'Bearer Token' as the auth type. The response is a 403 Forbidden status with a 56 ms response time and 1.14 KB of data. The response body is JSON:

```
1 {
2   "success": false,
3   "message": "Security violation: Potential malicious input detected."
4 }
```

13.6 XSS Attack

Using Script to Bypass Input data validation

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A POST request is configured to 'http://localhost:5000/api/alerts'. The body is set to 'raw' and contains the following JSON payload:

```
1 {
2   "title": "<script>alert('XSS')</script>",
3   "severity": "high",
4   "description": "XSS test payload"
5 }
```

The response is a 403 Forbidden status, indicating that the request was blocked due to the detected malicious input.

14 Time Based Blind SQL

Is an attacking technique **manipulate the database query's response time** to extract information.

Request Example

The screenshot shows a REST client interface with a collection named 'analyst'. A GET request is configured to 'http://localhost:5000/api/alerts?id=1 AND SLEEP(5)'. The authorization is set to 'Bearer Token'. The response is a 200 OK status, indicating that the request was successful despite the time-based attack payload.

Quick Detection:

Response Time	What It Means	Action
< 200ms	Normal	Monitor
> 5s - 5000ms+	Confirmed attack	Emergency response

Response time: 290ms (normal)

15. Roles & Permissions Summary

Role	Permissions
<code>soc_manager</code>	Full access — create, read, update, delete all resources including user management, blocking, and report generation
<code>Viewer</code>	Read-only — can view alerts, incidents, dashboard, and own profile. Cannot create, update, delete, or trigger any write operations
<code>Analyst</code>	Analyst can investigate alerts, update status, add comments, create incidents, and generate reports but cannot manage users or delete data (those are SOC Manager only)

16. Quick Reference — All Endpoints

Method	Endpoint	Description	Min Role
GET	/health	Health check — no auth needed	All
POST	/api/auth/login	Login and get JWT token	All
POST	/api/auth/register	Register new user	All
GET	/api/auth/me	Get current user info	All
GET	/api/alerts	Get all alerts	All
POST	/api/alerts	Create a new alert	soc_manager
GET	/api/alerts/stats	Get alert statistics by severity	soc_manager
PUT	/api/alerts/{alert_id}/acknowledge	Acknowledge an alert	soc_manager
GET	/api/incidents	Get all incidents	All
POST	/api/incidents	Create a new incident	soc_manager
PUT	/api/incidents/{incident_id}	Update an incident	soc_manager
DELETE	/api/incidents/{incident_id}	Delete an incident	soc_manager
GET	/api/dashboard/summary	Dashboard summary metrics	All
GET	/api/dashboard/recent-alerts	Recent alerts for dashboard	All
GET	/api/users	List all users	soc_manager
POST	/api/users	Create a new user	soc_manager
POST	/api/threat-intel/scan	Scan an IOC	soc_manager
GET	/api/threat-intel/recent	Get recent threat IOCs	soc_manager

POST	/api/actions/block	Block a malicious entity	soc_manager
GET	/api/geo/{ip}	Geo-lookup for a single IP	soc_manager
POST	/api/geo/bulk	Bulk geo-lookup for multiple IPs	soc_manager
GET	/api/reports/summary	JSON summary report	soc_manager
GET	/api/reports/alerts/pdf	Download PDF alerts report	soc_manager
GET	/api/reports/alerts/csv?type=alerts&format=csv	Download CSV alerts report	soc_manager